

## Day 8: Guidance, Solvers & Fast Sampling

---

- **Guidance, Solvers & Fast Sampling** — Controlling and accelerating diffusion
- 3 hours lecture + practical
- Slides, notes, and code on the course site

- Conditional Generation & Guidance
- Sampling = Solving a Differential Equation
- Fast High-Order Solvers
- Few-Step Sampling

## 1 · Conditional Generation & Guidance

---

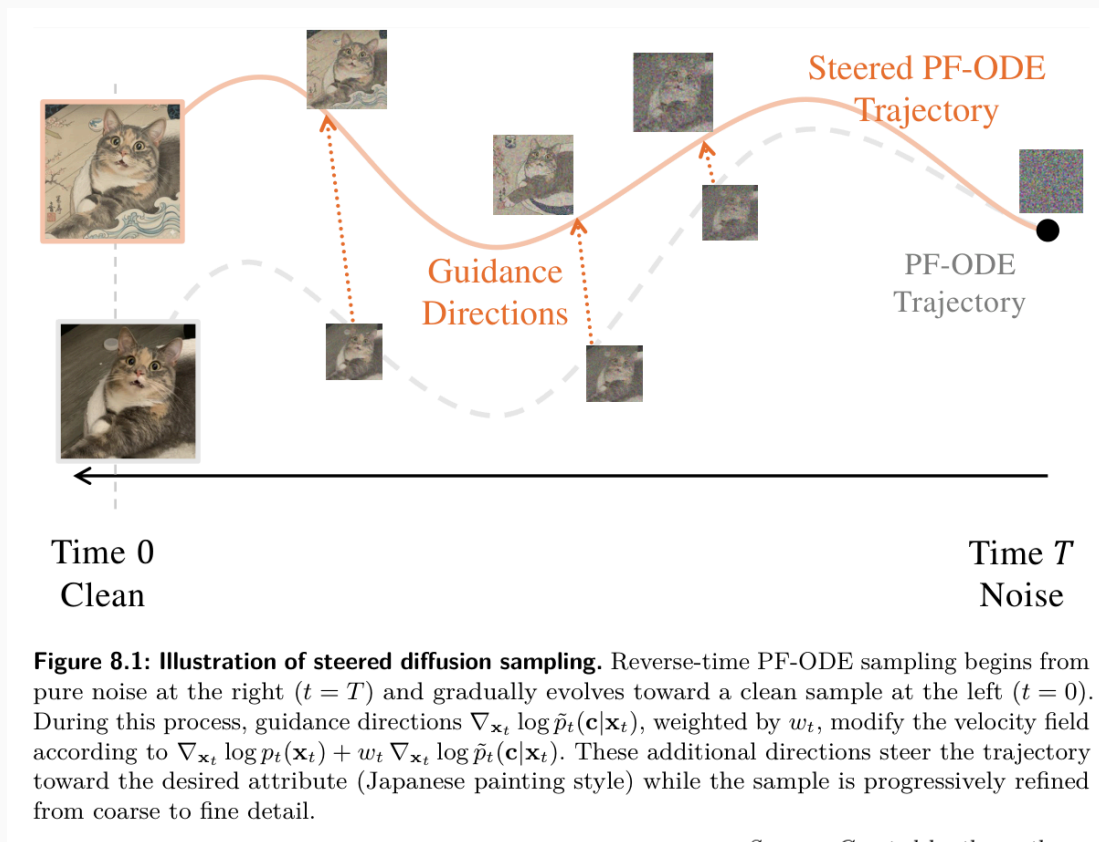
# 1.1 Conditioning a Diffusion Model

- Goal: sample  $p(x \mid c)$  — class, text prompt, image
- Train the denoiser with the condition:  $\varepsilon_{\theta}(x_t, t, c)$
- Conditional score  $\nabla \log p_t(x \mid c)$  steers sampling
- But naive conditioning often **under-uses** the condition
- Guidance amplifies the influence of  $c$

## 1.2 Classifier Guidance

- Bayes:  $\nabla \log p(x|c) = \nabla \log p(x) + \nabla \log p(c|x)$
- Train a classifier  $p_\varphi(c | x_t)$  on noisy inputs
- Push samples toward high  $p(c|x)$  using its gradient
- Guidance scale  $w$ :  $\nabla \log p + w \nabla \log p(c|x)$
- Needs a separate noisy-data classifier (a drawback)

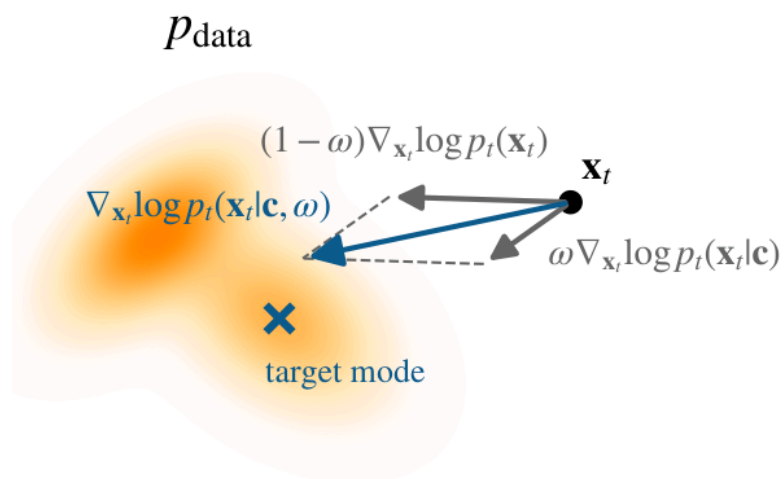
## 1.2 Classifier Guidance



## 1.3 Classifier-Free Guidance

- Train one net for both conditional & unconditional (drop  $c$ )
- Combine at sampling time, no external classifier:
- $\tilde{\varepsilon} = (1 + w)\varepsilon_{\theta}(x, t, c) - w\varepsilon_{\theta}(x, t, \emptyset)$
- $w$  trades diversity (low) vs fidelity/prompt-match (high)
- The workhorse of text-to-image models

## 1.3 Classifier-Free Guidance



**Figure 8.2: Illustration of CFG.** The adjusted score  $\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t | \mathbf{c}, \omega)$  is obtained as a linear interpolation between the unconditional score  $\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t)$  and the conditional score  $\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t | \mathbf{c})$ , weighted by  $\omega$ . The resulting direction steers samples from the prior toward modes of the data distribution consistent with the target condition.

## 1.4 Derivation: Guidance from Bayes

- Bayes:  $p(x|c) \propto p(x)p(c|x)$
- Take  $\nabla_x \log$ : scores add
- Implicit classifier:  $\nabla \log p(c|x) = \nabla \log p(x|c) - \nabla \log p(x)$
- Score form  $\rightarrow$  noise form via  $s = -\varepsilon \frac{\nabla}{\sigma_t}$
- Sharpen: raise  $p(c|x)$  to power  $w \rightarrow$  guidance scale

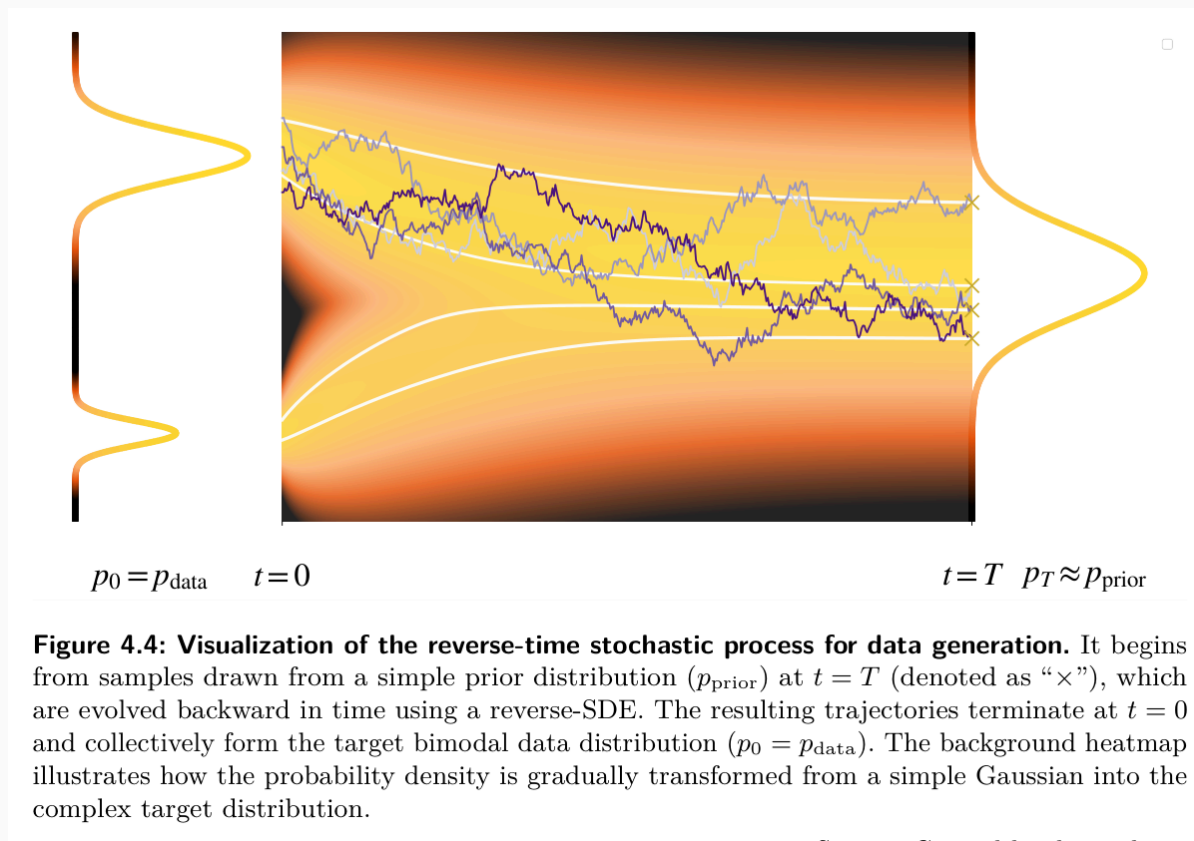
2 · Sampling = Solving a Differential Equation

---

## 2.1 Sampling Is Numerical Integration

- Day 7: sampling = integrate the reverse SDE or PF-ODE
- Each step needs one network eval (NFE) of the score
- Quality vs cost = solver accuracy vs number of steps
- Discretize time  $T = t_0 > t_1 > \dots > t_N = 0$
- Better solver  $\rightarrow$  same quality in fewer steps

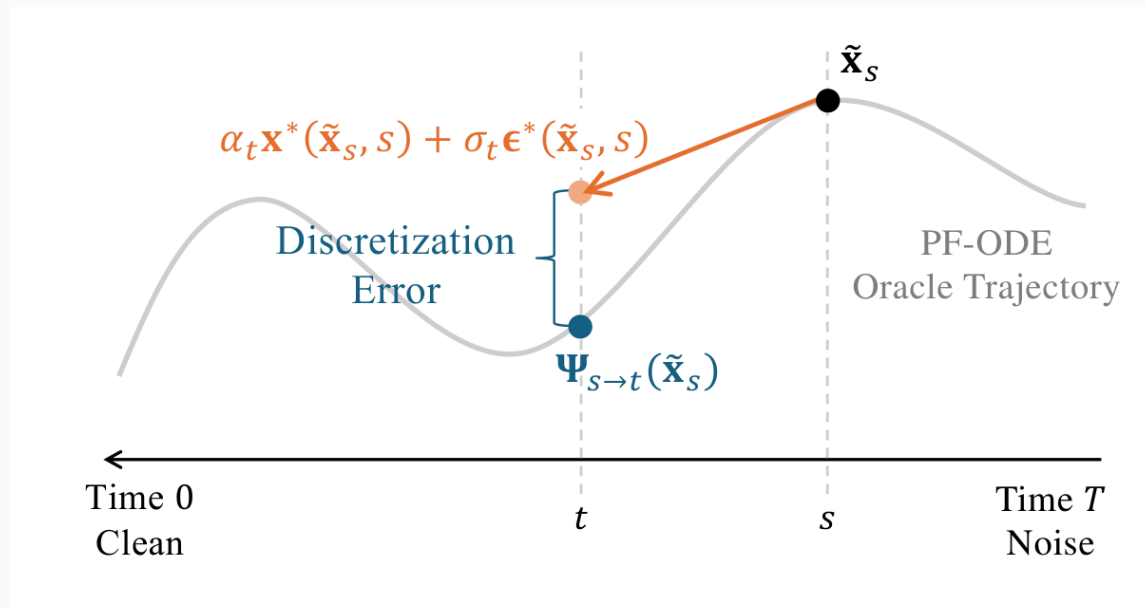
## 2.1 Sampling Is Numerical Integration



## 2.2 Derivation: PF-ODE from the FPE

- Start from FPE of forward SDE:  $\partial_t p_t = -\nabla \cdot (\mu p_t) + \dots$
- Split diffusion term with product rule; log-deriv. trick on  $\nabla p_t$
- Factor out  $p_t$ :  $\partial_t p_t = -\nabla \cdot (\tilde{\mu} p_t)$
- $\tilde{\mu} = \mu - \frac{1}{2} \nabla [\sigma^2] - \frac{1}{2} \sigma^2 \nabla \log p_t$
- Zero diffusion  $\rightarrow$  Liouville eqn  $\rightarrow$  ODE  $dx = \tilde{\mu} dt$
- Same marginals; score coefficient  $\frac{1}{2}$  not 1 (vs reverse SDE)

## 2.2 Derivation: PF-ODE from the FPE



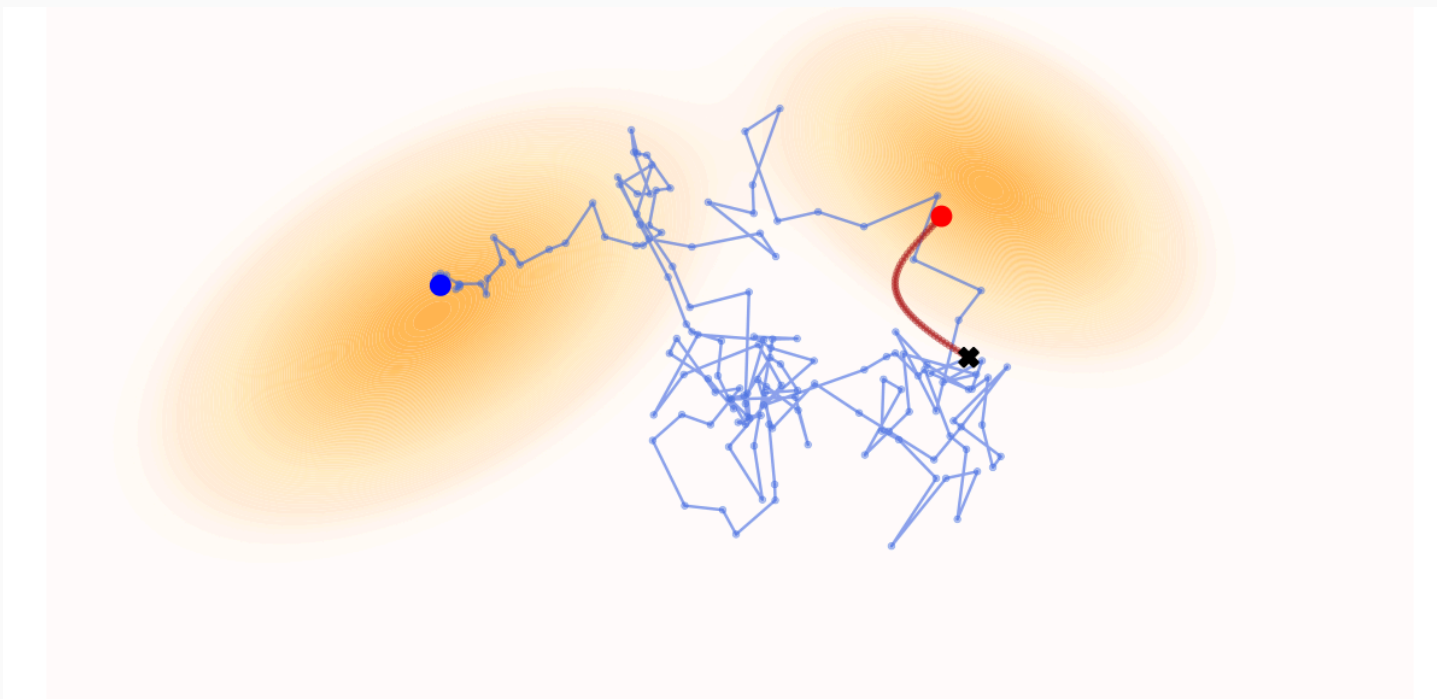
**Figure 9.1: Illustration of DDIM as an Euler discretization of the PF-ODE.** Starting from a state  $\tilde{\mathbf{x}}_s$  at time  $s$ , the oracle PF-ODE trajectory (gray curve) deterministically evolves to  $\Psi_{s \rightarrow t}(\tilde{\mathbf{x}}_s)$  at time  $t$ . In contrast, the DDIM update (orange) directly maps  $\tilde{\mathbf{x}}_s$  to  $\alpha_t \mathbf{x}^*(\tilde{\mathbf{x}}_s, s) + \sigma_t \epsilon^*(\tilde{\mathbf{x}}_s, s)$ . The discrepancy between this Euler step and the true PF-ODE trajectory introduces a discretization error, shown in blue. If  $t$  is far from  $s$ , the discrepancy can become large, leading to degraded generation quality.

Source: Created by the author.

## 2.3 DDIM as Euler on the PF-ODE

- DDIM = deterministic sampler = Euler on the PF-ODE
- Non-Markovian: skip steps without retraining
- Same trained model, far fewer steps than DDPM
- Deterministic  $\rightarrow$  reproducible, invertible, editable
- First-order: error per step  $O(\Delta t^2)$

## 2.3 DDIM as Euler on the PF-ODE



**Figure 4.7: (2D) Illustration of sampling from the Score SDE.** The forward SDE is designed as  $d\mathbf{x}(t) = \sqrt{2t} d\mathbf{w}(t)$ , with drift  $\mathbf{f} \equiv \mathbf{0}$  and diffusion coefficient  $g(t) = \sqrt{2t}$  on  $[0, T]$ , evolves a two-mode Gaussian mixture  $p_0 = p_{\text{data}}$  into  $p_T \approx p_{\text{prior}} := \mathcal{N}(\mathbf{0}, T^2 \mathbf{I})$ . Shown are sampling trajectories

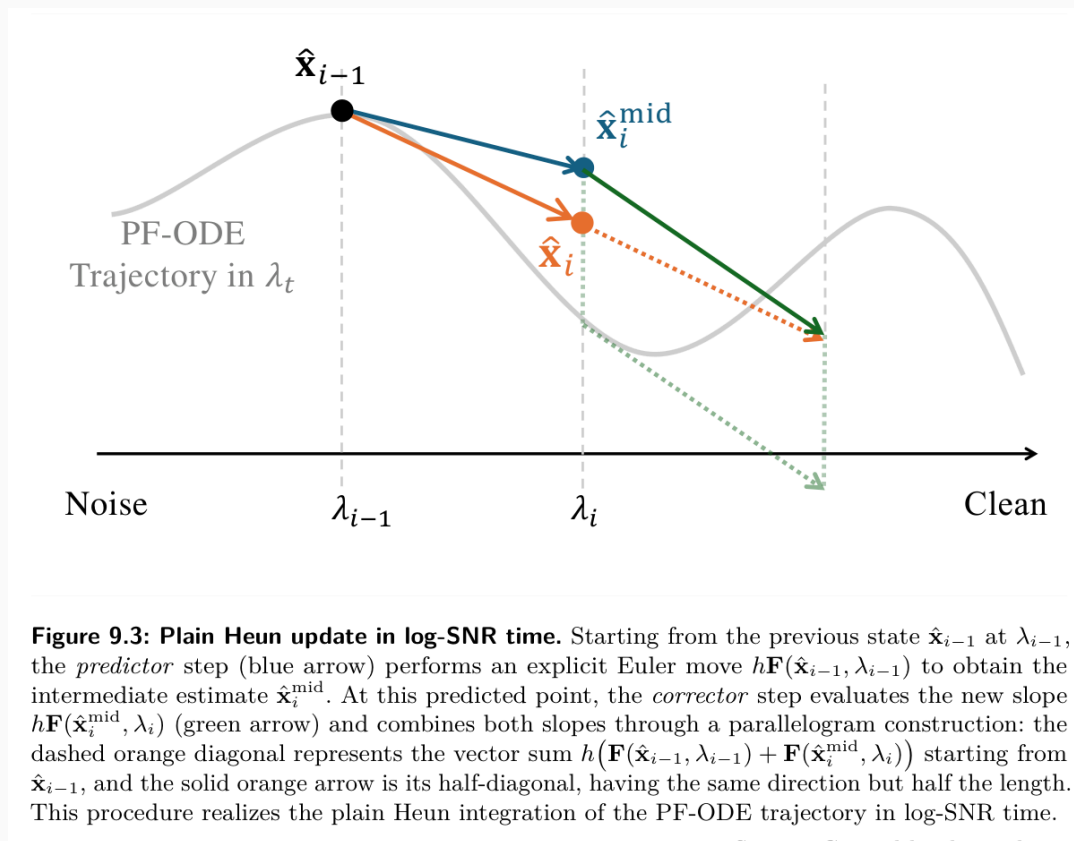
## 2.4 Discretization Error & Step Count

- Few steps + 1st-order  $\rightarrow$  visible artifacts
- Error accumulates over the trajectory
- More steps shrink error but cost more NFEs
- Curved trajectories are harder to integrate
- Two fixes: better solver, or straighter paths

## 2.5 Stochastic vs Deterministic Samplers

- SDE samplers inject noise each step (self-correcting)
- ODE samplers are deterministic (fast, fewer steps)
- SDE: better at high NFE; ODE: better at low NFE
- Churn: add a little noise to an ODE solver (EDM)
- Fokker-Planck: both share the same marginals  $p_t$

## 2.5 Stochastic vs Deterministic Samplers



### 3 · Fast High-Order Solvers

---

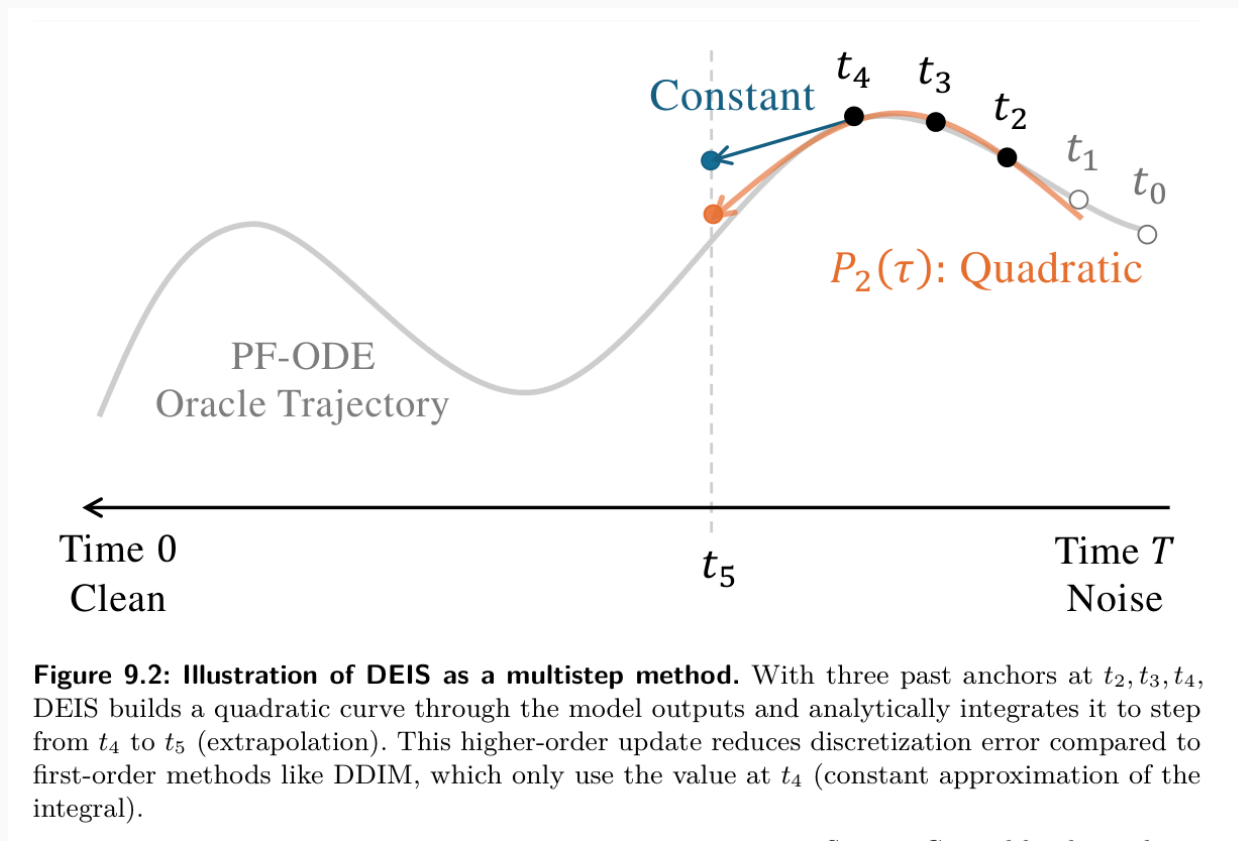
## 3.1 Higher-Order: Heun's Method

- Euler uses the slope at the start of the step
- Heun averages start & end slopes (predictor-corrector)
- 2nd-order: error per step  $O(\Delta t^3)$
- 2 NFEs/step but far fewer steps overall
- Backbone of the EDM sampler

## 3.2 The Right Clock: log-SNR Time

- Solver accuracy depends on the time variable
- Step uniformly in log-SNR  $\lambda = \log\left(\alpha^2 \frac{1}{\sigma^2}\right)$
- Spends steps where the trajectory bends most
- EDM uses a tailored  $\sigma$  schedule (same idea)
- Good schedule  $\rightarrow$  big quality gain for free

## 3.2 The Right Clock: log-SNR Time



**Figure 9.2: Illustration of DEIS as a multistep method.** With three past anchors at  $t_2, t_3, t_4$ , DEIS builds a quadratic curve through the model outputs and analytically integrates it to step from  $t_4$  to  $t_5$  (extrapolation). This higher-order update reduces discretization error compared to first-order methods like DDIM, which only use the value at  $t_4$  (constant approximation of the integral).

### 3.3 Exponential Integrators (DPM-Solver / DEIS)

- PF-ODE = linear drift + nonlinear network term
- Solve the linear part **exactly** (integrating factor, Day 1)
- Only approximate the smooth network part
- Multistep: reuse past evals for higher order
- 10-20 NFEs for high quality (DPM-Solver, DEIS)

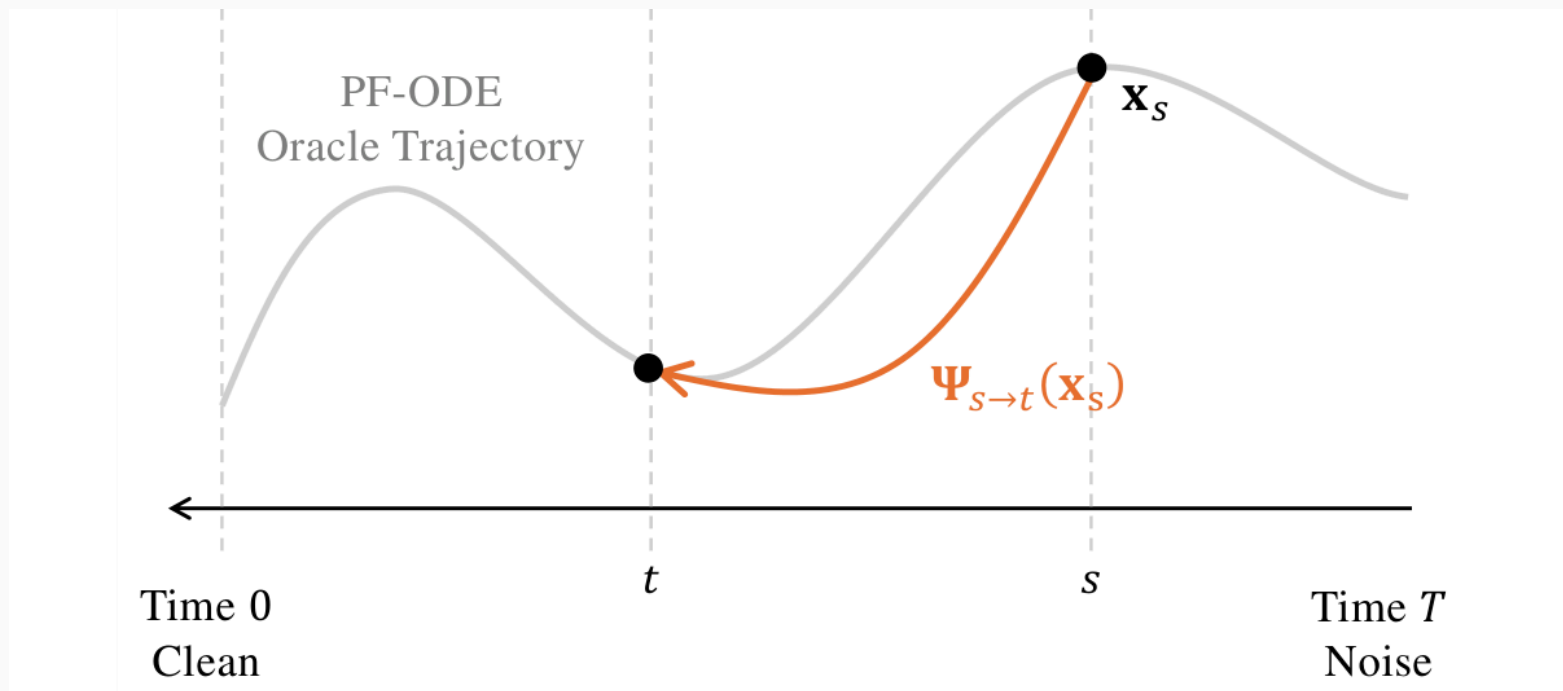
## 4 · Few-Step Sampling

---

## 4.1 The Bottleneck: Many Function Evals

- Even good solvers need 10-50 NFEs
- Real-time / interactive needs 1-4 steps
- Idea: learn to **jump** across time, not crawl
- Distill a slow teacher into a fast student
- Flow maps formalize the jump

## 4.1 The Bottleneck: Many Function Evals



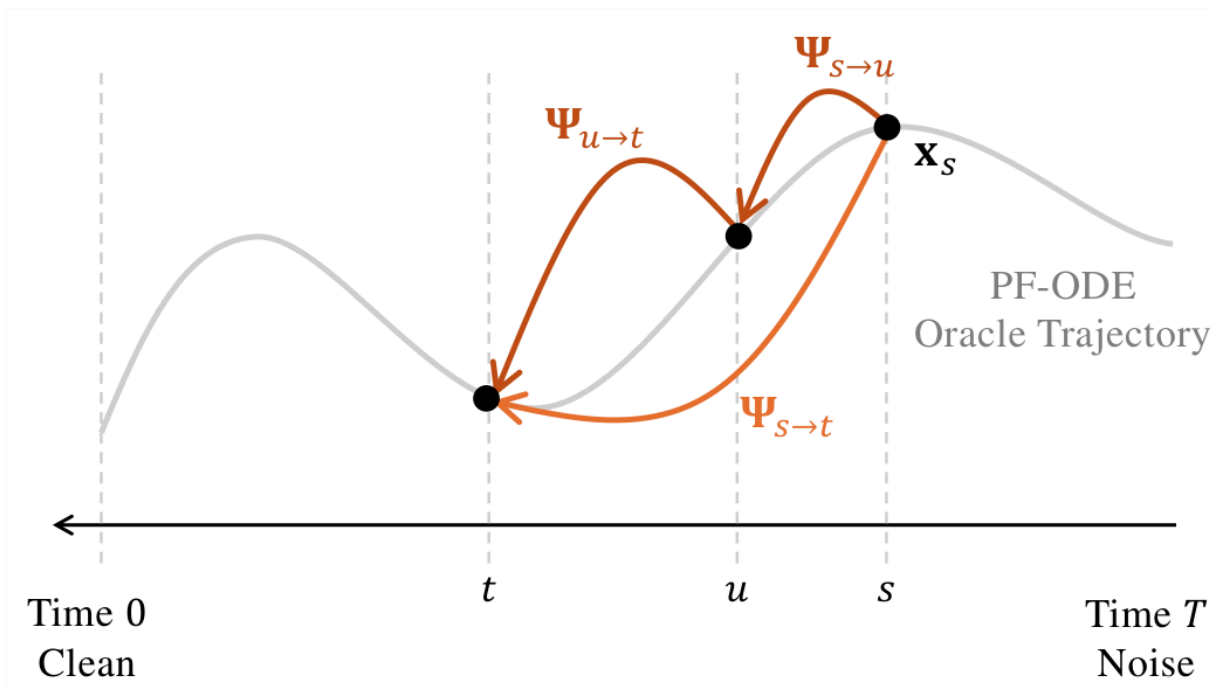
**Figure 11.2: Illustration of the flow map.** Starting from any state  $\mathbf{x}_s$  at time  $s$ , the flow map  $\Psi_{s \rightarrow t}$  transports it to the corresponding point on the same PF-ODE oracle trajectory at time  $t$ .

*Source: Created by the authors.*

## 4.2 Flow Maps

- Flow map  $\Phi_{s \rightarrow t}$ : jump a sample from time  $s$  to  $t$
- Integrates the ODE in **one** learned step
- $x_t = \Phi_{s \rightarrow t}(x_s)$  for any  $s, t$
- Generalizes the single-step generator
- Learn it by distillation or self-consistency

## 4.2 Flow Maps



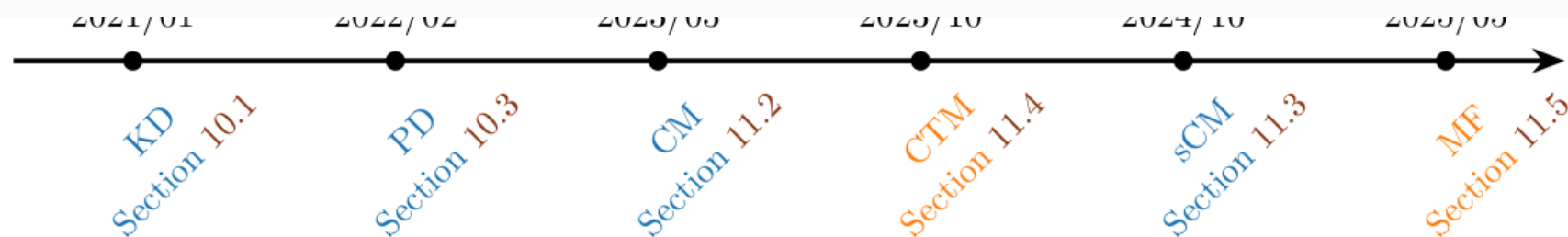
**Figure 11.3: Illustration of the flow map semigroup property.** This property states that transitioning from  $s$  to  $u$  and then from  $u$  to  $t$  is equivalent to transitioning directly from  $s$  to  $t$ .

*Source: Created by the author.*

## 4.3 The Semigroup Property

- Composing jumps = one bigger jump:
- $\Phi_{s \rightarrow t} = \Phi_{u \rightarrow t} \circ \Phi_{s \rightarrow u}$
- Consistency: all points on a trajectory map to the same  $x_0$
- Self-consistency gives a training signal w/o a teacher
- Enables 1-step generation

## 4.3 The Semigroup Property



**Figure 11.1: Timeline of Flow Map Modeling.** We use **blue** for the special case  $\Psi_{s \rightarrow 0}$  and **orange** for the general map  $\Psi_{s \rightarrow t}$ .

*Source: Created by the authors*

## 4.4 Consistency & Distillation

- Consistency models: train  $f_\theta(x_t, t) \approx x_0$  for all  $t$
- Distillation: match a multi-step teacher in few steps
- 1-4 step sampling, near teacher quality
- Trade a little quality for huge speedups
- Active frontier: real-time generative models

- Day 8: **Guidance, Solvers & Fast Sampling**
- Controlling and accelerating diffusion
- Questions welcome — practical follows

## Questions?

Practical session follows — see course site.